

Tarea 4: CRUD Agenda de Contactos

Programacion Avanzada

Unidad II - PHP y MySQLi con Prepared Statements

1. Objetivo

Desarrollar un modulo CRUD completo (Create, Read, Update, Delete) para una Agenda de Contactos utilizando PHP con MySQLi y consultas preparadas (prepared statements). El alumno debera aplicar los mismos patrones y estructura utilizados en el proyecto Gestor Bibliotecario.

2. Base de Datos

Crea una base de datos llamada agenda_contactos y ejecuta el siguiente script SQL para crear la tabla contactos:

```
CREATE DATABASE agenda_contactos;
USE agenda_contactos;

CREATE TABLE contactos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    apellido VARCHAR(100) NOT NULL,
    telefono VARCHAR(20),
    correo VARCHAR(150),
    direccion TEXT,
    categoria VARCHAR(50) DEFAULT 'General',
    fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

3. Estructura del Proyecto

Organiza tu proyecto dentro de la carpeta agenda_contactos con la siguiente estructura:

```
agenda_contactos/
|
+-- config/
|   +-- config.php
|   +-- connection.php
|
+-- includes/
|   +-- header.php
|   +-- footer.php
|
+-- contactos/
    +-- index.php      (Listar contactos)
    +-- create.php     (Formulario de nuevo contacto)
    +-- save.php       (Guardar nuevo contacto)
    +-- edit.php       (Formulario de edicion)
    +-- update.php     (Actualizar contacto)
```

+-- delete.php (Eliminar contacto)

NOTA: Puedes reutilizar los archivos includes/header.php y includes/footer.php del proyecto Gestor Bibliotecario, solo ajusta los enlaces de navegacion.

4. Archivos de Configuración

config/config.php

```
<?php
$db_host = "localhost";
$db_user = "root";
$db_pass = "";
$db_name = "agenda_contactos";
?>
```

config/connection.php

```
<?php
require_once __DIR__ . "/config.php";
$conn = new mysqli($db_host, $db_user, $db_pass, $db_name);
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
$conn->set_charset("utf8");
?>
```

5. Especificaciones del CRUD

5.1 CREATE - Nuevo Contacto (create.php + save.php)

El formulario de create.php debe contener los siguientes campos: Nombre (obligatorio), Apellido (obligatorio), Telefono, Correo electronico, Direccion y Categoria (con opciones como: General, Familiar, Trabajo, Amigos).

En save.php debes recibir los datos via POST, sanitizarlos y usar una consulta preparada:

```
$stmt = $conn->prepare("INSERT INTO contactos (nombre, apellido, telefono, correo, direccion, categoria)
$stmt->bind_param("ssssss", $nombre, $apellido, $telefono, $correo, $direccion, $categoria);
if ($stmt->execute()) {
    header("Location: index.php?success=" . urlencode("Contacto creado exitosamente"));
}
```

5.2 READ - Listar Contactos (index.php)

En index.php debes mostrar todos los contactos en una tabla HTML con las columnas: #, Nombre, Apellido, Telefono, Correo, Categoria, Fecha de Registro y Acciones. Usa una consulta preparada con SELECT y fetch_assoc():

```
$stmt = $conn->prepare("SELECT id, nombre, apellido, telefono, correo, categoria, fecha_registro FROM c
$stmt->execute();
$resultado = $stmt->get_result();
while ($row = $resultado->fetch_assoc()) {
    echo "<tr>";
    echo "<td>" . htmlspecialchars($row["id"]) . "</td>";
```

```

        echo "<td>" . htmlspecialchars($row["nombre"]) . "</td>";
        // ... resto de columnas
    }

```

5.3 READ - Obtener un Contacto (edit.php)

En edit.php debes obtener el contacto por su ID usando una consulta preparada con WHERE, y mostrar un formulario precargado con los datos existentes.

```

$stmt = $conn->prepare("SELECT * FROM contactos WHERE id = ?");
$stmt->bind_param("i", $id);
$stmt->execute();
$resultado = $stmt->get_result();
$contacto = $resultado->fetch_assoc();

```

5.4 UPDATE - Actualizar Contacto (update.php)

Recibe los datos via POST (incluyendo el ID oculto) y actualiza el registro con una consulta preparada:

```

$stmt = $conn->prepare("UPDATE contactos SET nombre = ?, apellido = ?, telefono = ?, correo = ?, direccion = ?, categoria = ? WHERE id = ?");
$stmt->bind_param("ssssssi", $nombre, $apellido, $telefono, $correo, $direccion, $categoria, $id);
if ($stmt->execute()) {
    header("Location: index.php?success=" . urlencode("Contacto actualizado exitosamente"));
}

```

5.5 DELETE - Eliminar Contacto (delete.php)

Recibe el ID por GET y elimina el contacto usando una consulta preparada:

```

$stmt = $conn->prepare("DELETE FROM contactos WHERE id = ?");
$stmt->bind_param("i", $id);
if ($stmt->execute()) {
    header("Location: index.php?success=" . urlencode("Contacto eliminado exitosamente"));
}

```

6. Requisitos Tecnicos

- Todas las consultas SQL deben usar prepared statements con bind_param().
- Sanitizar todas las entradas del usuario con trim(), intval() y el operador ??.
- Sanitizar todas las salidas HTML con htmlspecialchars() para prevenir XSS.
- Validar que los campos obligatorios (nombre y apellido) no esten vacios.
- Manejar mensajes de exito y error via parametros GET (success / error).
- Usar la misma estructura de conexion y estilo del proyecto Gestor Bibliotecario.
- Aplicar disenio responsivo con Bootstrap (puedes copiar el CSS del gestor).

7. Rubrica de Evaluacion

Criterio	Puntos
Conexion a BD y configuracion correcta	1
CREATE - Formulario y prepared statement	1.5
READ - Listado con tabla y fetch_assoc()	1.5
READ - Obtener contacto por ID (edit.php)	1
UPDATE - Prepared statement con WHERE	1.5

DELETE - Prepared statement con WHERE	1
Sanitizacion de entradas (trim, intval, ??)	1
Sanitizacion de salidas (htmlspecialchars)	0.5
Mensajes de exito/error y validaciones	0.5
Diseno y estructura del proyecto	0.5

Total: 10 puntos

8. Fecha y Forma de Entrega

- Fecha limite: Revisar calendario de la materia en el salon virtual.
- Forma de entrega: Subir el proyecto comprimido en ZIP al salon virtual.
- El archivo ZIP debe llamarse: T4_AgendaContactos_[Matricula].zip
- Incluir el script SQL de la base de datos dentro del ZIP.
- Adjunta capturas de pantalla del CRUD funcionando (al menos 3).